

EXPERIMENT 11

Name-Atharva Prabhu

Class-D20A

Rollno-45

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Hyperledger Fabric is a **permissioned blockchain platform** developed by the Linux Foundation.

Unlike public blockchains, it is designed for **enterprise use**, where participants are known and trusted.

It provides:

- High scalability
- Privacy and confidentiality
- Modular architecture

Key Components of Fabric Network

a) Peers

- Nodes that maintain the ledger and execute transactions
- Two types:
 - **Endorsing peers** → simulate transactions
 - **Committing peers** → validate and commit transactions

b) Orderer

- Responsible for **ordering transactions** into blocks
- Ensures consistency across the network

c) Ledger

Each peer maintains a ledger with:

- **Blockchain** → immutable transaction log
- **World State** → current state of assets (stored in DB like CouchDB)

d) Channels

- Private communication layer between organizations
- Ensures **data isolation and confidentiality**

e) Membership Service Provider (MSP)

- Manages identities and authentication using digital certificates.

Chaincode

Chaincode is the **business logic** of the blockchain.

It defines:

- Rules for asset creation
- Validation of transactions
- Query and update operations

Chaincode is typically written in:

- Go
- JavaScript (Node.js)
- Java

Chaincode Lifecycle

Steps involved:

1. **Package** → Convert code into deployable format
2. **Install** → Install on peers
3. **Approve** → Organizations approve chaincode definition

4. **Commit** → Chaincode becomes active on channel

Transaction Flow in Fabric

1. Client sends transaction proposal
2. Endorsing peers simulate and sign results
3. Client sends endorsed transaction to orderer
4. Orderer creates block
5. Peers validate and commit to ledger

Procedure:

Prerequisite-Install WSL

Command:

wsl --install

```
spandan@LAPTOP-9JIMM8I3: ~
Ubuntu has been installed.
Launching Ubuntu...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows user name.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: spandan
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Apr  9 14:40:23 UTC 2026

System load:  0.34               Processes:            47
Usage of /:   0.1% of 1006.85GB   Users logged in:     0
Memory usage: 14%               IPv4 address for eth0: 192.168.69.236
Swap usage:   0%

This message is shown once a day. To disable it please create the
/home/spandan/.hushlogin file.
spandan@LAPTOP-9JIMM8I3:~$
```

Step 1-Update the system

Command:

`sudo apt update`

`sudo apt upgrade -y`

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1881 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [343 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
```

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  ethtool libllvm20 mesa-libgallium
The following packages have been kept back:
  libgl1-amber-dri libglapi-mesa
The following packages will be upgraded:
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps
```

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt install jq curl git nodejs npm -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ docker --version
Docker version 29.3.1, build c2be9cc
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ node -v
v18.19.1
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ npm -v
9.2.0
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
```

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (159/159), done.
remote: Compressing objects: 100% (101/101), done.
Receiving objects: 55% (8615/15618), 13.93 MiB | 887.00 KiB/s
```

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /mnt/c/Users/HOME/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0  0 --:--:-- --:--:-- --:--:--    0
 86 120M   86 104M    0     0 3094k    0  0:00:40  0:00:34  0:00:06 3290k
```

```

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
d41d93850539: Pull complete
eb2bee6aa5bd: Pull complete
dbac02a8c9ad: Pull complete
f8c55d757df2: Pull complete
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
b1cba2e842ca: Already exists
94c191cca972: Pull complete
7f5f5563c22b: Pull complete
61c558a0dbbe: Pull complete
efe901203a90: Pull complete
e756cbd2164a: Pull complete
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15

```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```

spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      1aad8b995da9      156MB      0B
ghcr.io/hyperledger/fabric-ca:1.5.17          0548296e36d4      244MB      0B
ghcr.io/hyperledger/fabric-ccenv:2.5.15       f80b289a3828      671MB      0B
ghcr.io/hyperledger/fabric-orderer:2.5.15     4fb4e60fb623      118MB      0B
ghcr.io/hyperledger/fabric-peer:2.5.15        d5150827e5b1      154MB      0B
hyperledger/fabric-baseos:2.5                 1aad8b995da9      156MB      0B
hyperledger/fabric-baseos:2.5.15              1aad8b995da9      156MB      0B
hyperledger/fabric-baseos:latest              1aad8b995da9      156MB      0B
hyperledger/fabric-ca:1.5                     0548296e36d4      244MB      0B
hyperledger/fabric-ca:1.5.17                  0548296e36d4      244MB      0B
hyperledger/fabric-ca:latest                  0548296e36d4      244MB      0B
hyperledger/fabric-ccenv:2.5                  f80b289a3828      671MB      0B
hyperledger/fabric-ccenv:2.5.15               f80b289a3828      671MB      0B
hyperledger/fabric-ccenv:latest               f80b289a3828      671MB      0B
hyperledger/fabric-orderer:2.5                4fb4e60fb623      118MB      0B
hyperledger/fabric-orderer:2.5.15             4fb4e60fb623      118MB      0B
hyperledger/fabric-orderer:latest             4fb4e60fb623      118MB      0B
hyperledger/fabric-peer:2.5                   d5150827e5b1      154MB      0B
hyperledger/fabric-peer:2.5.15                d5150827e5b1      154MB      0B
hyperledger/fabric-peer:latest                d5150827e5b1      154MB      0B

```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ export PATH=$PATH:/mnt/c/Users/HOME/fabric-samples/bin
export FABRIC_CFG_PATH=/mnt/c/Users/HOME/fabric-samples/config
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running the peer version verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ cd test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
to from 'cryptogen'
```



```

LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/mnt/c/Users/HOME/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
  ✓Network fabric_test Created 2.7ss
  ✓Volume compose_orderer.example.com Created 2.6ss
  ✓Volume compose_peer0.org1.example.com Created 0.2ss
  ✓Volume compose_peer0.org2.example.com Created 0.7ss
  ✓Container orderer.example.com Started 67.8s
  ✓Container peer0.org2.example.com Started 66.6s
  ✓Container peer0.org1.example.com Started 68.5s
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
MES 246fe6a8d014 hyperledger/fabric-peer:latest "peer node start" About a minute ago Up 44 seconds 0.0.0
0.9051->9051/tcp 0.1.9051->9051/tcp 0.0.0.9045->9045/tcp 0.1.9045->9045/tcp

```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-ba
sic/chaincode-javascript -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...

```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"InitLedger","Args":[]}'
2026-04-09 17:50:03.612 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":500,"Color":"blue","ID":"asset2","Owner":"Tom","Size":10}]
```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset2"]}'  
{"AppraisedValue":500,"Color":"blue","ID":"asset2","Owner":"Tom","Size":10}
```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke ... CreateAsset

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"CreateAsset","Args":["asset1","car","blue","Tom","500"]}'
2026-04-09 18:08:34.702 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset1","Color":"car","Size":null,"Owner":"Tom","AppraisedValue":500}
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $PEER0_ORG2_CA \
--waitForEvent \
-c '{"function":"TransferAsset","Args":["asset2","Jerry"]}'
2026-04-09 18:23:54.916 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [9729dfdb66a176d035db287b9b104d582d2ff8822ee56b79d14499c4abeb7860] committed with status (VALID) at localhost:9051
2026-04-09 18:23:54.933 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [9729dfdb66a176d035db287b9b104d582d2ff8822ee56b79d14499c4abeb7860] committed with status (VALID) at localhost:7051
2026-04-09 18:23:54.934 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset2","Color":"Jerry","Size":null,"Owner":"Jerry","AppraisedValue":500}
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

`./network.sh down`

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org2.example.com      Removed
✓Container orderer.example.com         Removed
✓Container peer0.org1.example.com      Removed
✓Volume compose_orderer.example.com   Removed
✓Volume compose_peer0.org2.example.com Removed
✓Network fabric_test                   Removed
✓Volume compose_peer0.org1.example.com Removed
```

Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.